

Relay — Beta Setup Sheet

Getting five testers into their own accounts. Companion to the Beta Test Guide. Written 2026-08-19.

Short answer to the question that prompted this: **there is no beta code, and you do not need one.** Signing up at relaystandby.com is open to anyone — no invitation, no allowlist, no waiting list. A tester can have their own working account in about two minutes without anything from you.

What you *do* hand out is a different thing, and the difference is worth getting straight before you send anything, because sending the wrong one produces a confusing evening.

Two kinds of account, and they are not the same. A **claim code** puts someone into *your* circle as a recipient or verifier — it is about your plan, not about giving them Relay. An **owner account** is theirs, holds their own vault, and needs no code from anybody. Most testers should end up with both, and the product has a button that walks them from one to the other.

Part A — The two accounts

READ THIS ONCE AND THE REST IS MECHANICAL

	STANDBY ACCOUNT	THEIR OWN OWNER ACCOUNT
What it is	Their place in <i>your</i> circle — the thing that lets them be recognised on the day, as a recipient or a verifier	Their own Relay: their vault, their people, their rules
How they get it	You issue a claim code and hand it to them	They sign up at relaystandby.com. Nothing needed from you
Where	relaystandby.com/claim — type the code in	relaystandby.com → Start, or /start
Costs	Nothing, ever. It holds nothing of theirs	Nothing during beta — see Part B
What it tests	Do people actually accept when asked? The number the whole design rests on, currently zero	Does setup make sense to a stranger? The thing only a person who did not build it can answer

The bridge between them is already in the product

Once someone has claimed and is standing by for you, their dashboard shows a panel headed "*Who would step in for you?*" with a **Start your own plan** button. It takes them to the same signup everyone else uses.

That is the sequence I would use for this cohort, and it is worth doing in that order for a reason beyond convenience: it puts each tester through the invitation experience first — the one we most need measured — and then lets them convert themselves, which is a real conversion path we would like to know works.

Nobody has to do the second half. A verifier who only ever claims and answers one question is a perfectly good tester and gives us the Phase 0 number. Starting their own plan is the bonus round, and it is the part that produces

the sharpest feedback about onboarding.

Part B — "Can I get a code to enable the beta?"

NO — AND HERE IS WHAT EXISTS INSTEAD

There is no beta gate to unlock. Signup is open to the public today: no invite code, no allowlist, nothing to enable. The only thing standing between a tester and an account is the authenticator-app step.

What a free account already gets, during the beta

LIMIT	FREE TIER	ENOUGH FOR TESTING?
Vault items	10	Yes — the guide asks for three or four
Recipients	4	Yes
Verifiers	4	Yes — quorums here run at one or two
Can actually release	Yes	Yes — this is the important one

Releasing is deliberately switched on for free accounts during the beta. Payment works and is live, but enforcing it now would mean the beta could not exercise the one thing the product exists for. It is a dated decision that gets reviewed from 2026-10-01, not an oversight — so testers get the full experience without anyone paying anything, and nothing they do is a purchase.

The one real lever, if a tester wants to go further

If somebody gets enthusiastic and hits the ten-item cap, there is a by-hand grant that lifts every limit for them:

```
npx tsx --env-file=.env.local scripts/grant-founding-tier.ts <their-email>

# and to undo it
npx tsx --env-file=.env.local scripts/grant-founding-tier.ts <their-email> --revoke
```

It marks the grant as given rather than bought, writes it to their audit log, and refuses to touch anyone who has actually paid. **Use it only if someone asks** — hitting a limit and telling you about it is itself a useful finding, and pre-emptively lifting caps hides it.

Why the comp is marked. The one number this whole project is waiting for is somebody at arm's length choosing to pay. A hand-granted account sits in the same table as a real subscription and would be indistinguishable from one — so it is stamped twice, and a granted plan can never be mistaken later for evidence that somebody wanted this enough to buy it.

Part C — Your run sheet

STEVE · ABOUT TWENTY MINUTES FOR FIVE PEOPLE

1. **List the cohort.** Create `.relay-cohort.json` in the repo root — it is gitignored, because it holds real people's names and addresses. Doug is already in it.

```
[
  { "name": "Doug Fish", "email": "...", "type": "recipient",
    "role": "recipient", "arm": "owner" },
  { "name": "...", "email": "...", "type": "verifier", "arm": "owner" },
  { "name": "...", "email": "...", "type": "verifier", "arm": "email" }
]
```

Mix the two arms deliberately. `owner` means you deliver the code yourself; `email` means Relay sends it. Comparing those two is the entire point of the exercise — if everyone gets the same arm we learn nothing about which one works.

2. **Dry run.** `npm run invite:cohort` creates nothing and lists who it would create. Read the list.
3. **Sign in at relaystandby.com and copy your session cookie**, then `export RELAY_OWNER_COOKIE=...`. The script needs it and stores it nowhere.
4. **Commit.** `npm run invite:cohort -- --commit`. It creates each person, issues every code, captures them to `.relay-cohort-codes.json` (also gitignored) and prints a ready-to-send message per person.
5. **Write each code onto its card** in Part D, or just keep the file open — whichever suits.
6. **Deliver the owner-arm codes yourself.** Text, call, or hand over. Do not email those.
7. **Make the four-word call to each person once they have claimed.** People → the person → Verified? **Until you do, their answer counts for nothing** and every test in the guide will quietly fail to open anything.
8. **Check the numbers** with `npx tsx --env-file=.env.local scripts/phase0-report.ts` — issued, opened, claimed, per arm. That table is the actual output of this round.

Each claim code is readable exactly once. Only a hash is stored, so if you lose it the invitation has to be cancelled and reissued. The script exists mostly to stop that happening — it captures every code at the moment of issue, and that file is the only copy.

Part D — Tester cards

ONE PER PERSON · FILL IN THE CODE, THEN HAND OVER OR READ OUT

The codes are deliberately not printed here. Three reasons, and any one of them is enough: they do not exist until you run the script; each is readable only once, so a printed copy would be the only copy; and this document lives in a public repository, where a claim code is a live credential that would let a stranger take somebody's place in your circle. **Write them in by hand, or keep `.relay-cohort-codes.json` open beside you.**

Doug Fish — recipient

Email

Delivery arm

owner (you tell him)

claim code — write it here

they enter it at relaystandby.com/claim

☐ code delivered

☐ claimed

☐ four-word call made

Tester 2 — verifier

Name

Email

Delivery arm

claim code — write it here

they enter it at relaystandby.com/claim

☐ code delivered

☐ claimed

☐ four-word call made

Tester 3 — verifier

Name

Email

Delivery arm

claim code — write it here

they enter it at relaystandby.com/claim

☐ code delivered ☐ claimed ☐ **four-word call made**

Tester 4 — recipient or verifier

Name

Email

Delivery arm

claim code — write it here

they enter it at relaystandby.com/claim

☐ code delivered ☐ claimed ☐ **four-word call made**

Tester 5 — spare

Name

Email

Delivery arm

claim code — write it here

they enter it at relaystandby.com/claim

☐ code delivered ☐ claimed ☐ **four-word call made**

Part E — Hand this bit to each tester

EVERYTHING THEY NEED, IN ONE PAGE

Getting started with Relay — two minutes

- 1. Take the place I have saved for you.** Go to relaystandby.com/claim and type in the short code I gave you. That is it — you now have a free account. It holds nothing of yours and costs nothing, ever.
 - 2. Set up a passkey when it offers.** Face, fingerprint or your device PIN. It means a new phone will not lock you out and there is no code to keep track of.
 - 3. Expect a phone call from me.** I will read out four words. Check they match the four on your screen. Thirty seconds, and it is the step that makes your answer count for anything later.
 - 4. Optional, and the most useful thing you could do:** on your standby screen there is a panel headed "*Who would step in for you?*" with a **Start your own plan** button. Press it and set Relay up for yourself. You do not need anything from me — no code, no invitation, no payment. Everything is free during the beta, including the part that actually opens access.
- Then tell me what was confusing.** That is genuinely the job. If a screen did not make sense, or a word was jargon, or you were not sure what you had just agreed to — that is the finding, and being blunt about it helps far more than being kind about it.

A note on links. Relay never sends a message that signs you in by clicking. You always type a short code into a page you went to yourself. If you get something claiming to be Relay with a "click here to access" link in it, it is not from us — and telling me you got one is a genuinely useful thing to report.

If someone is on Outlook, Hotmail, Live or MSN

Warn them in advance that Relay's mail often lands in junk there. It is a known, measured problem and it is not their mail client being broken. Ask them to look, and to tell you where it landed — that is data we cannot get any other way.

Part F — Quick reference

COMMANDS, IN ONE PLACE

WANT TO...	DO THIS
See who the cohort would create	<code>npm run invite:cohort</code>
Actually create them and get the codes	<code>npm run invite:cohort -- --commit</code>
See how many claimed, per arm	<code>npx tsx --env-file=.env.local scripts/phase0-report.ts</code>
Lift every limit for one tester	<code>npx tsx --env-file=.env.local scripts/grant-founding-tier.ts</code> <code><email></code>
Undo that	...same, with <code>--revoke</code>
Check nothing was left behind afterwards	<code>npm run verify:orphans</code>

Where things live

<code>.relay-cohort.json</code>	who to invite — you write this. Gitignored: real names and addresses
<code>.relay-cohort-codes.json</code>	the issued codes — the script writes this, and it is the only copy. Gitignored
<code>docs/Relay-Beta-Test-Guide.pdf</code>	the 23-page guide: what to test and what to report
<code>relaystandby.com/help</code>	the full user guide, for anyone who wants more detail on a screen

The one thing to get right, restated because it is the one that will cost you an evening: make the four-word call to every verifier before you run any of the tests. An unverified verifier can press Confirm, the screen will thank them, and the release will not open — with nothing anywhere saying why.